

Manuel d'Administration

SangoMarket by Legos — Guide complet de gestion de la plateforme

Table des matières

1. Démarrage de la plateforme

2. Configuration de l'environnement

3. Base de données

4. Gestion des utilisateurs

5. Gestion des commandes

6. Gestion des paiements / Escrow

7. Gestion des produits

8. Gestion des transporteurs
9. Points de retrait

10. Sécurité

11. Logs et monitoring

12. Déploiement en production

13. Internationalisation

14. Tableau de bord admin

15. Dépannage courant

16. Contacts

1. Démarrage de la plateforme

Prérequis

- Docker + Docker Compose installés
- Node.js 20+ (développement local uniquement)
- PostgreSQL 16 (si sans Docker)

Lancement avec Docker (recommandé)

```
# Depuis /home/legos/Documents/ProjetPerso/
docker-compose up -d

# Vérifier que tout tourne
docker-compose ps

# Consulter les logs
docker-compose logs -f backend
docker-compose logs -f frontend
docker-compose logs -f postgres
```

Service	URL	Conteneur
Frontend	http://localhost:3000	diasporamarket_web
API Backend	http://localhost:4000	diasporamarket_api
Base de données	localhost:5432	diasporamarket_db

Health Check

http://localhost:4000/health

—

Lancement en développement local

```
# Terminal 1 – Backend
cd backend && npm install && cp .env.example .env
npm run dev    # ts-node-dev, rechargement automatique

# Terminal 2 – Frontend
cd frontend && npm install
npm run dev    # Next.js dev server
```

2. Configuration de l'environnement

Fichier backend/.env

Variable	Valeur par défaut	Description
PORT	4000	Port du serveur API
NODE_ENV	development	development / production
DATABASE_URL	postgresql://postgres:password123@localhost:5432/diasporamarket	URL PostgreSQL
JWT_SECRET	⚠ À changer	Clé secrète JWT, minimum 32 caractères
JWT_EXPIRES_IN	7d	Durée de vie des tokens
FRONTEND_URL	http://localhost:3000	CORS — URL du frontend
ORANGE_MONEY_CLIENT_ID	—	Clé API Orange Money (production)
ORANGE_MONEY_CLIENT_SECRET	—	Secret API Orange Money
MAX_FILE_SIZE	5242880	5 Mo max par image uploadée

ADMIN_EMAIL

admin@diasporamarket.cf

Email
administrateur

Fichier frontend/.env.local

```
NEXT_PUBLIC_API_URL=http://localhost:4000/api  
# Production : remplacer par l'URL publique du backend
```

3. Base de données

Connexion directe

```
# Via Docker  
docker exec -it diasporamarket_db psql -U postgres -d diasporamarket  
  
# En local  
psql -U postgres -d diasporamarket
```

Initialisation du schéma

```
cd backend  
psql -U postgres -d diasporamarket -f src/config/schema.sql  
npm run migrate # via le script de migration  
npm run seed # données de test
```

Tables principales

Table	Rôle
users	Tous les comptes (clients, vendeurs, transporteurs, admins)
products	Catalogue de produits avec pays d'origine
orders	Commandes clients
order_items	Lignes de commande (produit, quantité, prix)
escrow_transactions	Paielements en séquestre (Orange Money / cash)
transporters	Profils des transporteurs/GP
trips	Voyages annoncés par les transporteurs
pickup_points	Points de retrait à Bangui
product_reviews	Avis sur les produits (rating 1-5)
transporter_reviews	Avis sur les transporteurs
notifications	Notifications utilisateurs

Requêtes SQL utiles

```
-- Lister tous les utilisateurs
SELECT id, name, email, role, is_verified, created_at FROM users ORDER BY created_at DESC;

-- Promouvoir un utilisateur en admin
UPDATE users SET role = 'admin' WHERE email = 'exemple@email.com';

-- Vérifier un transporteur manuellement
UPDATE transporters SET is_verified = true WHERE id = 'UUID_DU_TRANSPORTEUR';

-- Commandes en attente de libération escrow
SELECT o.order_number, o.total_amount, o.status, e.status as escrow_status
FROM orders o
JOIN escrow_transactions e ON e.order_id = o.id
WHERE o.escrow_released = false AND o.status = 'pret_retrait';

-- Chiffre d'affaires mensuel
SELECT DATE_TRUNC('month', created_at) as mois,
       SUM(total_amount) as ca_total
FROM orders WHERE payment_status = 'libere'
GROUP BY mois ORDER BY mois DESC;

-- Produits par pays d'origine
SELECT origin_country, COUNT(*) as nb_produits
FROM products WHERE is_available = true
GROUP BY origin_country ORDER BY nb_produits DESC;
```

4. Gestion des utilisateurs

Rôles disponibles

Rôle	Description
client	Acheteur — navigue, commande, donne des avis
vendeur	Vendeur — crée produits, gère commandes reçues, accès dashboard stats
transporteur	Logisticien GP — publie des voyages, accès profil transporteur
admin	Accès complet — libère escrow, rembourse, gestion totale

Créer un compte admin

Il n'existe pas de route d'inscription admin publique. Deux méthodes :

```
-- Option A : directement en base (générer un hash bcrypt cost 12)
INSERT INTO users (id, name, email, password_hash, role, country_code, is_verified)
VALUES (gen_random_uuid(), 'Nom Admin', 'admin@sangomarket.com',
       '$2a$12$HASH_BCRYPT_ICI', 'admin', 'CF', true);

-- Option B : s'inscrire normalement puis promouvoir
UPDATE users SET role = 'admin' WHERE email = 'votre@email.com';
```

Suspendre / réactiver un compte

```
UPDATE users SET is_active = false WHERE id = 'UUID'; -- Suspension
UPDATE users SET is_active = true WHERE id = 'UUID'; -- Réactivation
```

5. Gestion des commandes

Cycle de vie d'une commande

```
en_attente → paye → en_preparation → expedie → arrive_bangui → pret_retrait → recupere
                                     ↘ annule
```

Mettre à jour le statut (API)

```
curl -X PATCH http://localhost:4000/api/orders/ORDER_ID/status \
-H "Authorization: Bearer ADMIN_TOKEN" \
-H "Content-Type: application/json" \
-d '{"status": "arrive_bangui"}'
```

6. Gestion des paiements / Escrow

Fonctionnement du système escrow

Étape	Statut escrow	Description
1. Initiation paiement	en_attente	Client initie le paiement Orange Money ou cash
2. Paiement confirmé	bloque	Argent bloqué, vendeur peut expédier
3. Client récupère	bloque	Commande marquée "pret_retrait" → "recupere"
4. Libération	libere	Admin/vendeur libère l'argent au vendeur
Alt. Remboursement	rembourse	Admin rembourse le client si problème

```
# Libérer l'escrow
curl -X POST http://localhost:4000/api/payments/release/ORDER_ID \
-H "Authorization: Bearer ADMIN_TOKEN"

# Rembourser
curl -X POST http://localhost:4000/api/payments/refund/ORDER_ID \
-H "Authorization: Bearer ADMIN_TOKEN"
```

Surveiller les escrows bloqués

```
SELECT o.order_number, u.name AS client, o.total_amount,
       e.reference, e.created_at AS bloque_le
FROM escrow_transactions e
JOIN orders o ON o.id = e.order_id
JOIN users u ON u.id = o.client_id
```

```
WHERE e.status = 'bloque'
ORDER BY e.created_at ASC;
```

7. Gestion des produits

```
# Créer un produit
curl -X POST http://localhost:4000/api/products \
  -H "Authorization: Bearer TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "title": "Tissu wax premium",
    "price": 15000, "currency": "XAF",
    "category": "mode", "originCountry": "SN", "stock": 50
  }'

# Désactiver un produit
curl -X PATCH http://localhost:4000/api/products/PRODUCT_ID \
  -H "Authorization: Bearer ADMIN_TOKEN" \
  -d '{"isAvailable": false}'

# Upload d'image (max 5 Mo)
curl -X POST http://localhost:4000/api/uploads \
  -H "Authorization: Bearer TOKEN" \
  -F "image=@/chemin/image.jpg"
# Retourne: { "url": "/uploads/filename.jpg" }
```

8. Gestion des transporteurs

```
-- Vérifier un transporteur en base
UPDATE transporters SET is_verified = true WHERE id = 'UUID';

-- Désactiver un voyage GP
curl -X PATCH http://localhost:4000/api/trips/TRIP_ID \
  -H "Authorization: Bearer ADMIN_TOKEN" \
  -d '{"isActive": false}'
```

9. Points de retrait (Bangui)

```
-- Ajouter un nouveau point de retrait
INSERT INTO pickup_points (id, name, address, city, phone, opening_hours, is_active)
VALUES (
  gen_random_uuid(),
  'Nouveau Point Retrait',
  'Avenue des Martyrs, Bangui',
  'Bangui', '+23672XXXXXX', 'Lun-Sam 8h-18h', true
);
```

10. Sécurité

Rate Limiting

Endpoint	Limite
Global (toutes routes)	200 requêtes / 15 min / IP
/api/auth/*	10 requêtes / 15 min / IP

Invalider tous les tokens JWT (en cas de compromission)

```
# Changer le JWT_SECRET dans backend/.env, puis redémarrer
docker-compose restart backend
```

Sauvegarde base de données

```
# Backup
docker exec diasporamarket_db pg_dump -U postgres diasporamarket > backup_$(date +%Y%m%d).sql

# Restaurer
docker exec -i diasporamarket_db psql -U postgres diasporamarket < backup_20260101.sql
```

11. Logs et monitoring

```
# Logs en temps réel
docker-compose logs -f

# Backend seulement
docker-compose logs -f backend

# Health check
curl http://localhost:4000/health
# Réponse : { "status": "ok", "timestamp": "...", "uptime": 123 }
```

12. Déploiement en production

⚠ Variables critiques à changer impérativement avant la mise en production.

```
NODE_ENV=production
JWT_SECRET=<secret_aléatoire_64_chars_minimum>
DATABASE_URL=<url_postgresql_production>
FRONTEND_URL=https://sangomarket.com
ORANGE_MONEY_CLIENT_ID=<production_key>
ORANGE_MONEY_CLIENT_SECRET=<production_secret>
```

Build production

```
cd backend && npm run build # génère ./dist/
cd frontend && npm run build # génère ./next/standalone/
```

Configuration Nginx (reverse proxy)

```
server {
    listen 443 ssl;
    server_name sangomarket.com;
    location / { proxy_pass http://localhost:3000; }
}
server {
    listen 443 ssl;
    server_name api.sangomarket.com;
    location / { proxy_pass http://localhost:4000; }
}
```

13. Internationalisation

Code	Langue	Statut
fr	Français	Défaut
en	English	Disponible
es	Español	Disponible
pt	Português	Disponible
de	Deutsch	Disponible

Pour ajouter une langue, éditer `frontend/lib/i18n/translations.ts` et `LanguageSwitcher.tsx`.

14. Tableau de bord admin

Accès : se connecter avec un compte **admin** puis naviguer vers `/dashboard/admin`.

- Vue globale des utilisateurs (clients, vendeurs, transporteurs)
- Gestion des commandes et mise à jour des statuts
- Libération / remboursement des escrows
- Vérification des profils transporteurs
- Statistiques générales de la plateforme

15. Dépannage courant

Backend ne démarre pas

```
docker-compose logs postgres
docker exec diasporamarket_db pg_isready -U postgres
docker-compose exec backend env | grep DATABASE
```


Erreur CORS

Vérifier que `FRONTEND_URL` dans `.env` correspond exactement à l'origine du frontend (avec `http://` et sans slash final).

Réinitialiser la base de données

⚠ ATTENTION : cette opération supprime toutes les données.

```
docker-compose down -v
docker-compose up -d
docker-compose exec backend npm run seed
```

Token JWT refusé

- Vérifier que le client envoie `Authorization: Bearer <token>`
- Vérifier que le token n'est pas expiré (7 jours par défaut)
- Vérifier que `JWT_SECRET` n'a pas changé depuis l'émission du token

16. Contacts et support

Champ	Valeur
Email administrateur	Idirose29@gmail.com
Projet	SangoMarket by Legos
Répertoire	/home/legos/Documents/ProjetPerso
Date du document	Avril 2026